Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

ЗВІТ

з лабораторної роботи № 4
з дисципліни «Методи та засоби продуктової аналітики»

**Тема: «AI-підкріплена дата-аналітика:
побудова власного аналітичного асистента»**

Виконав:
студент групи ІМ-31
Соломка Андрій

Київ — 2026

Мета роботи

Побудувати робочий прототип AI-асистента продуктового аналітика — інструмент, який забирає на себе рутинну частину роботи з даними: профілювання нового датасету, пошук сегментів з аномальною поведінкою, відповіді на разові питання та підготовку тексту для стейкхолдера. Окремим завданням є критичне ставлення до виходу LLM: інструмент має не лише користуватися моделлю, а й контролювати її.

Теоретичні відомості

Чому LLM не можна доручати розрахунки

Мовна модель не виконує арифметику — вона продовжує послідовність токенів за найімовірнішим продовженням. Коли модель «рахує» середній чек, вона насправді генерує число, яке виглядає правдоподібно для такого контексту. Для аналітики це критично: помилка на 10% у числі виглядає так само впевнено, як і правильне значення, і не супроводжується жодним сигналом невпевненості.

Звідси випливає архітектурний принцип, на якому побудований увесь інструмент: усі числа рахує код, модель лише інтерпретує готові числа. Модель жодного разу не отримує сирий CSV — вона бачить JSON із уже порахованими фактами. Це звужує простір, у якому вона може помилитися, до формулювань.

Галюцинації та їх перевірка

Навіть отримавши готові факти, модель може назвати число, якого в них немає: перемішати два показники, округлити в інший бік або просто вставити правдоподібне значення. Найнебезпечніший випадок — не очевидна дурниця, а число поруч із правильним (3,7% проти 4,1%), яке не викликає підозри і йде далі у звіт.

Тому промпт тут — не захист, а лише зменшення ймовірності. Поверх нього потрібна програмна перевірка: з тексту відповіді витягуються всі числа і звіряються з множиною чисел, які реально є у фактах. Складність у тому, що збіг має бути не точним: модель законно пише «139,8 тис.» замість 139 793 або «4,32%» замість 0,0432, тому перевірка враховує округлення та перерахунок у тисячі й відсотки.

Локальні мовні моделі

Модель не обов'язково викликати через хмарний API. Відкриті моделі з вагами у вільному доступі (Gemma, Llama, Qwen) запускаються на власному комп'ютері через LM Studio, Ollama чи llama.cpp. Ці застосунки піднімають локальний сервер з тим самим інтерфейсом, що й OpenAI Chat Completions, тому код клієнта змінюється лише адресою сервера. Переваги для аналітики — дані не залишають комп'ютер, запити нічого не коштують і не залежать від ліміту ключа. Недолік — мала модель (кілька мільярдів параметрів у 4-бітному квантуванні) помітно гірше тримає складні інструкції, тому для неї схема відповіді й програмна перевірка ще важливіші, ніж для великої хмарної.

Структуровані виходи

Витягувати числа і смислові блоки з вільного тексту ненадійно: формат відповіді змінюється від виклику до виклику. Тому всі функції інструменту вимагають від моделі JSON за заданою схемою. На рівні API це підкріплюється режимом JSON (response_format), а якщо локальний сервер цей режим не підтримує, запит повторюється без нього й JSON витягується з тексту відповіді.

Безпека згенерованого коду

Функція «питання природною мовою → код» означає виконання коду, який написала модель. Це класична вразливість: достатньо, щоб у даних або в питанні опинилася інструкція, і модель згенерує звернення до файлової системи чи мережі. Тому згенерований код проходить чорний список небезпечних конструкцій і виконується з урізаним набором вбудованих функцій та на копії датафрейму.

Окремо варто розрізняти два джерела помилок. Промпт зменшує кількість поганих генерацій, але не гарантує їх відсутності; перевірка коду ловить те, що прослизнуло. Обидва механізми потрібні одночасно саме тому, що закривають різні ризики.

Декомпозиція зміни метрики

Щоб пояснити, чому метрика змінилась, недостатньо показати її динаміку. Потрібен внесок кожного сегмента: наскільки змінилось значення всередині групи і яку частку загальної зміни ця група дала. Внесок рахується

як відношення зміни в сегменті до загальної зміни; сума внесків дорівнює 100%, а сегменти, що рухались у протилежний бік, дають від'ємний внесок. Саме ця таблиця й відповідає на питання «хто винен», а модель формулює гіпотезу причини вже поверх неї.

Хід роботи

Архітектура інструменту

Інструмент названо DataPilot і реалізовано як Python-пакет із CLI. Логіка всюди однакова: код рахує → модель пояснює → код перевіряє.

```
datapilot/
  llm.py          шар доступу до LLM: LM Studio / Ollama / Anthropic / OpenAI / mock
  profiling.py    детерміноване профілювання, прапорці якості даних
  insights.py     зрізи по сегментах, кореляції, денні аномалії за z-score
  eda.py          функція 1 — автоматичний EDA
  hypotheses.py   функція 2 — генерація гіпотез
  nl2pandas.py    функція 3 — NL -> pandas + пісочниця виконання
  anomaly.py      функція 4 — пояснення зміни метрики
  summary.py      функція 5 — executive summary
  verify.py       власна функція — звірка чисел моделі з фактами
  cli.py          інтерфейс командного рядка
data/             демонстраційний датасет
demo/             ноутбук із прогоном і збережені виводи команд (demo/outputs/local — на Gemma)
tests/            тести на ризиковані місця
```

Лістинг 1 — структура репозиторію

Шар доступу до моделі винесено окремо, щоб інструмент не був прив'язаний до одного провайдера: усі функції викликають complete_json, яка повертає вже розпарсений словник. Основний режим — локальна модель: якщо в .env нічого не задано, інструмент сам перевіряє, чи запущено LM Studio (порт 1234) або Ollama (порт 11434), і звертається до знайденого сервера. Хмарні Anthropic та OpenAI підключаються ключем у .env, а коли немає ні сервера, ні ключа, вмикається режим mock — він дозволяє запускати тести й демонструвати детерміновану частину без моделі.

Локальна модель

Для роботи використано LM Studio 0.4.28 і модель Gemma 4 E4B від Google (gemma-4-e4b-it-mlx, формат MLX для Apple Silicon, 4-бітне квантування, 6,9 ГБ) — ту саму, що розгорталась локально в лабораторній з основ веб-технологій. Модель завантажується з контекстом 16 384 токени, щоб у нього вміщався JSON із фактами, і віддається через вбудований сервер LM Studio:

```
$ lms load gemma-4-e4b-it-mlx --context-length 16384
$ lms server start
Success! Server is now running on port 1234

# .env — необов'язковий, сервер знаходиться і без нього
DATAPILOT_PROVIDER=lmstudio
DATAPILOT_MODEL=gemma-4-e4b-it-mlx
```

Лістинг 2 — запуск локальної моделі

Клієнт — офіційна бібліотека openai з адресою http://localhost:1234/v1; ключ локальний сервер не перевіряє. Один виклик на ноутбуці з Apple Silicon

триває 20–60 секунд залежно від обсягу фактів. Усі виводи нижче, позначені як відповіді моделі, отримано саме від Gemma 4 E4B.

Демонстраційні дані

Інструмент тестувався на навчальній базі мобільного застосунку за 5 квітня – 11 травня 2024 року — тій самій, що у лабораторній №3. З сирих таблиць (понад 120 МБ) зібрано три аналітичні зрізи: users.csv (60 000 користувачів із реєстрацією, платформою, країною, A/B-групою, агрегованою активністю та виручкою), daily_metrics.csv (7 825 рядків «день × платформа × країна») та payments.csv (11 330 платежів). Датасет обрано свідомо: у ньому є реальна аномалія — реєстрації на iOS зупиняються після 20 квітня, — на якій видно, чи справді працює функція пояснення аномалій.

Функція 1. Автоматичний EDA

Профілювання повністю детерміноване: типи колонок, пропуски, унікальні значення, квартилі, викиди за правилом 1,5·IQR, дублікати рядків. Поверх цього працюють евристики якості даних — і це теж код, а не модель.

```
def quality_flags(df: pd.DataFrame, profile: list[dict]) -> list[str]:
    """Евристики якості даних. Їх перевіряє код, а не модель."""
    flags = []
    n = len(df)
    dup = int(df.duplicated().sum())
    if dup:
        flags.append(f"Повні дублікати рядків: {dup} ({round(dup / n * 100, 2)}%)")
    for p in profile:
        if p["missing_pct"] > 20:
            flags.append(f"Колонка '{p['column']}': {p['missing_pct']}% пропусків")
        if p["kind"] == "categorical" and p["unique"] == 1:
            flags.append(f"Колонка '{p['column']}' має одне унікальне значення — марна для аналі
зу")
        if p["kind"] == "numeric" and p.get("zeros_pct", 0) > 60:
            flags.append(f"Колонка '{p['column']}': {p['zeros_pct']}% нулів — перевірте, чи це н
е пропуски")
        if p["kind"] == "numeric" and p.get("outliers_iqr", 0) > 0:
            share = round(p["outliers_iqr"] / n * 100, 2)
            if share > 5:
                flags.append(f"Колонка '{p['column']}': {p['outliers_iqr']} викидів за IQR ({sha
re}%)")
        if p["kind"] == "numeric" and p.get("min", 0) < 0 and any(
                w in p["column"].lower() for w in ("cnt", "count", "amount", "revenue", "duratio
n", "age")):
            flags.append(f"Колонка '{p['column']}' має від'ємні значення, хоча за змістом не пов
инна")
    if not flags:
        flags.append("Грубих проблем якості не виявлено")
    return flags
```

Лістинг 3 — евристики якості даних (фрагмент profiling.py)

Модель отримує JSON профілю і повертає інтерпретацію за фіксованою схемою: огляд, проблеми якості, ключові спостереження, наступні кроки. Прапорець --no-llm вимикає звернення до моделі й лишає тільки розрахунки. Фрагмент відповіді Gemma на daily_metrics.csv:

```
Інтерпретація LLM
Проаналізовано дані за період з 05 квітня по 11 травня, що охоплює 37 унікальних днів. Датасет м
   істить інформацію про активність користувачів та фінансові показники, згруповані за різними п
   латформами та країнами.

Якість даних
 • У колонці 'revenue_usd' 81.97% значень дорівнює нулю, що потребує уточнення щодо того, чи це
   означає відсутність транзакцій, а не пропуски даних.
 • Колонки 'sessions', 'session_minutes', 'active_users', 'new_users', 'revenue_usd', 'payers',
   'activities' та 'avg_session_min' містять значну кількість викидів, які виявлені за методом I
   QR.
 • Колонка 'app' має найбільшу частку даних, що належать до платформи 'mobile' (55.16%), але так
   ож є значна частка даних від інших платформ.
 • Хоча всі колонки не містять пропущених значень (missing_pct = 0.0), висока відсоткові частки
   нулів у фінансових метриках можуть бути як бізнес-правилом, так і ознакою проблеми з логуванн
   ям.

Ключові спостереження
 • Середня кількість сесій становить 192.675, але медіана значно нижча — лише 8.0, що свідчить п
   ро сильний перекіс розподілу в бік високих значень.
 • Середній час сесії становить 5797.18 (ймовірно, у секундах або іншому масштабі), тоді як меді
   ана — 249.6, що підтверджує високу варіативність у тривалості сесій.
 • Коли аналізувати дохід ('revenue_usd'), слід враховувати, що 81.97% записів мають дохід нуль,
    тоді як середній дохід становить 11.9808.
 • Найбільша частина активності припадає на мобільну платформу ('mobile'), яка забезпечує найбіл
   ьшу частку даних (55.16%).
 • Хоча в загальному є 7825 записів, унікальних значень для 'country_name' лише 196, що вказує н
   а географічну диверсифікацію бази користувачів.

Що робити далі
 • Необхідно провести глибокий аналіз викидів у кількісних колонках ('sessions', 'session_minute
   s' тощо), щоб визначити, чи слід їх обробляти (калібрувати, видаляти або залишати як є).
 • Критично важливо узгодити з бізнес-замовниками, чи означає нуль у 'revenue_usd' відсутність д
   оходу або це фактичний стан, який потрібно моделювати.
 • Проаналізувати залежність між 'app' та показниками конверсії (наприклад, чи є значно відмінно
   сті у середньому дохідному потенціалі між 'mobile' та іншими платформами).
 • Вивчити, як розподіл між країнами корелює з ключовими метриками (наприклад, чи є певні країни
    з вищим середнім чеком або більшою активністю).

✔ Перевірка чисел: усі 15 числа з тексту знайдено у фактах.
```

Лістинг 4 — інтерпретація EDA від локальної моделі

Перший прогін показав типову слабкість малої моделі: середнє avg_session_min вона назвала «5797 секунд (1.6 год)», виручку revenue_usd — «12 євро», а ще згадала «коефіцієнт відмов», якого в даних немає. Верифікатор спіймав лише 1.6: інші числа були справжні, хибними були одиниці. Тому до промпту додано два правила — одиниці беруться з суфікса колонки (_min, _usd, _cnt), а метрики, яких немає в JSON, не згадуються. Після цього обидві помилки зникли, і всі 15 чисел відповіді пройшли перевірку.

Функція 2. Генерація гіпотез

Щоб гіпотези були предметними, моделі недостатньо дати df.describe() — він описує колонки загалом і не показує різниці між сегментами. Тому код рахує зрізи: середні значення ключових метрик по кожній категорійній колонці, розкид між найкращою і найгіршою групою з відношенням, кореляції та денну динаміку з відхиленнями понад 2,5 сигми. Метрики для

зрізів обираються не підряд, а за пріоритетом — спершу гроші й утримання, інакше гіпотези виходять про вік і технічні лічильники.

У схемі відповіді підстава винесена в окреме обов'язкове поле evidence. Це головне рішення цієї функції: коли для числа є спеціальне місце, модель його заповнює, і гіпотезу стає можливо перевірити. Поле how_to_check змушує одразу думати про спростовність. Навіть мала модель стабільно заповнює це поле числами зі зрізів:

```
Гіпотези (6)

1. [high] Android користувачі генерують значно вищий середній дохід ($1.84) порівняно з Desktop
   (0.688), що може бути пов'язано з їхньою більшою часткою платіжних користувачів (11.37% проти
    6.18%).
   Підстава: android avg_revenue_usd: 1.8413, android avg_is_payer: 0.1137 vs desktop avg_revenu
   e_usd: 0.688, desktop avg_is_payer: 0.0618
   Чому важливо: Android є ключовим драйвером монетизації, і потрібно зрозуміти, що саме виклика
   є цю різницю в конверсії та дохідності.
   Як перевірити: Порівняти деталі транзакцій (тип покупок, середній чек) між Android та Desktop
    користувачами.

2. [high] Група 'not_in_test' має значно вищий середній дохід ($0.7608) та коефіцієнт платежів (
   0.0848), ніж група 'control' ($0.4032 / 0.0503), що може свідчити про ефект відбору або вплив
    тестування.
   Підстава: not_in_test avg_revenue_usd: 0.7608, not_in_test avg_payments_cnt: 0.0848 vs contro
   l avg_revenue_usd: 0.4032, control avg_payments_cnt: 0.0503
   Чому важливо: Це критично для оцінки ефективності будь-яких змін, оскільки 'not_in_test' демо
   нструє значно кращі показники монетизації.
   Як перевірити: Проаналізувати, які саме користувачі потрапили в групи 'control' та 'not_in_te
   st', щоб виключити зміщення (selection bias).
```

Лістинг 5 — перші гіпотези від Gemma 4 E4B на users.csv

Функція 3. Питання природною мовою → pandas

Модель отримує схему датафрейму — назви колонок, типи, діапазони числових і до десяти реальних значень категорійних. Останнє виявилось принциповим: без прикладів значень модель пише df[df.app == 'iOS'], тоді як у даних ios у нижньому регістрі, і запит мовчки повертає нуль рядків — відповідь є, виглядає правдоподібно і є неправильною.

```
def check_code(code: str) -> None:
    low = code.lower()
    for bad in FORBIDDEN:
        if bad in low:
            raise UnsafeCode(f"У згенерованому коді є заборонена конструкція: {bad!r}")
    if "result" not in code:
        raise UnsafeCode("Код не присвоює змінну `result`")


def run_code(code: str, df: pd.DataFrame):
    check_code(code)
    env = {"df": df.copy(), "pd": pd, "np": np}
    exec(compile(textwrap.dedent(code), "<datapilot>", "exec"), {"__builtins__": {
        "len": len, "range": range, "min": min, "max": max, "sum": sum,
        "round": round, "sorted": sorted, "abs": abs, "list": list, "dict": dict,
        "set": set, "str": str, "int": int, "float": float, "bool": bool,
        "enumerate": enumerate, "zip": zip, "print": lambda *a, **k: None,
    }}, env)
    return env.get("result")
```

Лістинг 6 — перевірка та виконання згенерованого коду

Код завжди показується користувачеві разом із поясненням і припущеннями, які зробила модель, — аналітик має перевіряти логіку, а не довіряти чорній скриньці. Якщо потрібної колонки немає, модель повертає code: null і пояснення; це краще, ніж вигадана колонка і KeyError.

```
Згенерований код
result = df.groupby('country_name')['amount_usd'].sum().nlargest(10)

Пояснення
Цей код групує дані за назвою країни, обчислює загальну виручку (суму 'amount_usd') для кожної к
   раїни, а потім сортує їх за спаданням і вибирає топ-10.
 • припущення: Виручка визначається сумою колонки 'amount_usd'.

Результат
                                                    amount_usd
country_name
```

Лістинг 7 — питання «Покажи топ-10 країн за виручкою» на локальній моделі

Функція 4. Пояснення зміни метрики

Команда explain порівнює два періоди і рахує внесок кожного сегмента у зміну. Нижче — прогін на демо-даних: притік нових користувачів після 20 квітня виріс у чотири рази, і саме на цьому тлі видно, що реєстрації на iOS впали практично до нуля.

```
Метрика new_users
Базовий період   2024-04-05..2024-04-19: 27,204.0
Порівнюваний     2024-04-20..2024-05-11: 112,589.0
Зміна: 85,385.0 (313.87%)

Внесок сегментів: app
  group    base  compare  change  change_pct  contribution_pct
 mobile 17096.0 111109.0 94013.0      549.91            110.10
    ios  8710.0     89.0 -8621.0      -98.98            -10.10
android   775.0     79.0  -696.0      -89.81             -0.82
desktop   623.0   1312.0   689.0      110.59              0.81
unknown     0.0      0.0     0.0         NaN              0.00

Внесок сегментів: country_name
                  group   base  compare  change  change_pct  contribution_pct
United States of America 5243.0  21991.0 16748.0      319.44             19.61
      Dominican Republic  903.0   8831.0  7928.0      877.96              9.29
            South Africa 2801.0  10681.0  7880.0      281.33              9.23
                    Peru  962.0   7689.0  6727.0      699.27              7.88
                 Ukraine 3738.0   8716.0  4978.0      133.17              5.83
                   Spain  916.0   5743.0  4827.0      526.97              5.65
```

Лістинг 8 — розрахункова частина команди explain для метрики new_users

Це і є той випадок, заради якого функція будувалась: загальне зростання маскує локальну поломку. У відсотках зміни iOS дає −98,98%, а його внесок у загальну зміну — −10,1% при +110,1% у mobile. Модель поверх цієї таблиці формулює гіпотезу, причому факти й припущення рознесені в схемі по різних полях:

```
Висновок LLM
Метрика нових користувачів зросла на 313.87% в порівнятному періоді, що майже повністю зумовлене
    мобільними додатками.

Головні драйвери
 • Сегмент 'mobile' (за типом додатку) згенерував найбільший абсолютний ріст (+94013.0) та внесо
   к (+110.1%).
 • Сегмент 'United States of America' (за країною) згенерував найбільший абсолютний ріст (+16748
   .0) та внесок (+19.61%).

Можливі причини (гіпотези)
 • [medium] Експлуатація успішної кампанії або вихідний сплеск у мобільному сегменті, який був з
   начно потужнішим порівняно з базовим періодом.
   Мобільний сегмент показав зростання на 549.91% (з 17096.0 до 111109.0).

Що перевірити далі
 • Аналізувати які саме дії чи канали привели до такого потужного зростання в сегменті 'mobile'.
 • Звернути увагу на падіння у сегментах 'ios' (-98.98%) та 'android' (-89.81%), щоб зрозуміти,
   чи це технічні проблеми або зміна поведінки користувачів.
 • Проаналізувати, чи є зростання в мобільних додатках репрезентативним для загального тренду, ч
   и це ізольований ефект.

✔ Перевірка чисел: усі 8 числа з тексту знайдено у фактах.
```

Лістинг 9 — пояснення від Gemma 4 E4B

У першій версії промпту мала модель описала лише зростання mobile і США, а падіння iOS не згадала, і частину тексту написала англійською. Після правила «окремо назви сегменти зі значним падінням, навіть якщо загалом метрика зросла» падіння iOS і Android модель згадує в кожному прогоні — у гіпотезі або серед того, що перевірити далі. Заодно виявилась вада верифікатора: «падіння на 98.98%» він позначав як вигадане, бо у фактах стоїть −98.98. Тепер числа звіряються за модулем — знак модель передає словами.

Функція 5. Executive summary

На вхід подається JSON з уже порахованими метриками, на вихід — короткий текст для керівника. Промпт забороняє жаргон аналітиків і вимагає структури «що сталось → чому це важливо → що робити», причому рекомендація має бути дією, а не спостереженням; поле owner у схемі додатково підштовхує до конкретики.

```
Executive summary
За період з 5 квітня по 11 травня ми залучили майже 140 тисяч нових користувачів, що принесло за
   гальний дохід у $93,75 тис. Серед цих нових користувачів лише 3,7% здійснили платіж. Середня
   сесія тривала близько 30 хвилин, а пікова щоденна активність досягла 33 тисяч користувачів.

Головне
 • Загальний дохід склав $93,75 тис. при 2218 платіжних користувачах.
 • Пікова щоденна активність досягла 33 тис. користувачів.
 • Найбільший дохід прийшов з України ($22,95 тис.), США ($19,14 тис.) та Південної Африки ($6,0
   5 тис.).

Ризики
 • Низький відсоток конверсії в платіж (3,7%) серед усіх нових користувачів.
 • Значна залежність від мобільної платформи (128 тис. нових користувачів прийшло через мобільни
   й додаток).

Рекомендації
 • Збільшити конверсію нових користувачів у платіжних клієнтів. — Незважаючи на високий рівень з
   алучення (140 тис. нових користувачів), низький відсоток платіжних дій обмежує загальний дохі
   д. [Продукт/Маркетинг]

✔ Перевірка чисел: усі 16 числа з тексту знайдено у фактах.
```

Лістинг 10 — executive summary для CEO від локальної моделі

Власна функція. Верифікація чисел

Шоста функція — та, якої немає у переліку завдання, але без якої решта ненадійна. Після кожної відповіді з тексту моделі витягуються всі числа і звіряються з числами, що реально є у фактах. Допускаються округлення (139 793 → «139,8 тис.»), перерахунок у відсотки (0,0432 → «4,32%») і знак, переданий словами («падіння на 98,98%» для −98,98); дрібні цілі, роки та номери пунктів ігноруються, щоб не засмічувати список.

```
ok    = "За період зареєструвалось 139 793 користувачі, виручка 93 749.58 $."
bad   = "За період зареєструвалось 139 793 користувачі, конверсія в платіж 4.1%."

OK                          <- За період зареєструвалось 139 793 користувачі, виручка 93 749.58
    $.
НЕПІДТВЕРДЖЕНО [4.1]        <- За період зареєструвалось 139 793 користувачі, конверсія в платі
   ж 4.1%.
```

Лістинг 11 — верифікатор відрізняє правильне число від правдоподібного вигаданого

Реальне значення конверсії у платіж — 3,7%. Число 4,1% правдоподібне і стоїть поруч із правильним, тому без автоматичної перевірки воно спокійно пішло б у звіт. У CLI такі значення виводяться окремим попередженням. На локальній моделі верифікатор довів свою користь на практиці: саме він показав, що в інтерпретації EDA є число, якого немає у фактах.

Що змінилось при переході на локальну модель

Таблиця 1 — Проблеми Gemma 4 E4B і що зроблено

Проблема	**Рішення**
eda: хвилини названо секундами, долари — євро	правило про одиниці з суфікса назви колонки
eda: згадано «коефіцієнт відмов», якого немає в даних	правило не згадувати метрик поза JSON

Проблема	Рішення
explain: пропущено падіння iOS, частина тексту англійською	правило окремо називати сегменти з падінням; усі поля українською
верифікатор не приймав «падіння на 98,98%» для −98,98	звірка чисел за модулем + тест
відповідь щоразу трохи різна	temperature 0,2; числа все одно проходять верифікатор

Тестування та крайні випадки

Тести покривають саме ризиковані місця, а не тривіальні функції:

— верифікатор ловить вигадане число і не чіпає округлене, переведене у відсотки чи з від'ємним знаком, переданим словами;
— пісочниця відхиляє імпорти, читання файлів і звернення до dunder-атрибутів;
— код без присвоєння result відхиляється як такий, що не відповідає контракту;
— виконання згенерованого коду не змінює вихідний датафрейм;
— профілювання правильно рахує пропуски й піднімає відповідний прапорець;
— декомпозиція знаходить обвал iOS у демонстраційних даних.

```
$ python -m pytest tests -q
.........                                                [100%]
9 passed in 0.31s
```

Лістинг 12 — прогін тестів

Окремо перевірено поведінку на крайніх випадках: порожній CSV (інструмент завершується з поясненням, а не зі стектрейсом), відсутність і локального сервера, і ключа (режим mock із підказкою запустити LM Studio), локальний сервер без підтримки JSON-режиму (повторний запит без нього), запит до неіснуючої колонки (модель повертає code: null, користувач бачить пояснення), а також небезпечний код, який відхиляється перевіркою.

Репозиторій і документація

Код, демонстраційні дані, ноутбук із прогоном (включно з живими відповідями локальної моделі) та збережені виводи команд зібрані в репозиторії. README містить опис, інструкцію запуску LM Studio з Gemma 4 E4B, перелік функцій і приклади використання з реальним виводом. Окремий файл prompts.md містить усі промпти з розбором: що робить кожне правило,

яку проблему воно закриває і чому формулювання саме таке, — зокрема правила, додані після прогону на малій моделі.

Репозиторій: https://github.com/AndriiSolomka/kpi-product-analytics-lab4

Висновки

У ході роботи побудовано DataPilot — CLI-асистента продуктового аналітика, який реалізує п'ять функцій із завдання (автоматичний EDA, генерацію гіпотез, запити природною мовою, пояснення аномалій та executive summary) і одну власну — верифікацію чисел у відповідях моделі. Інструмент працює на локальній моделі Gemma 4 E4B у LM Studio, без платного API і без передачі даних назовні.

Головний висновок стосується розподілу ролей. Спокуса віддати LLM весь аналіз велика, але модель не рахує — вона генерує правдоподібне. Робочою виявилась схема, в якій LLM не бачить сирих даних взагалі: pandas рахує факти, модель перетворює їх на текст, код перевіряє текст назад проти фактів. Саме тому вистачило малої моделі на 4 млрд параметрів: від неї потрібні лише формулювання, а не розрахунки.

Другий висновок — про промпти. Найбільший ефект дали не ввічливі прохання, а зміни структури відповіді: винесення підстави гіпотези в окреме обов'язкове поле, розділення фактів і припущень на різні поля схеми, дозвіл явно відповісти «не можу». Мала модель додатково потребувала явних правил там, де велика здогадується сама: про одиниці виміру й про сегменти, що впали на тлі загального зростання. Помилки в одиницях верифікатор чисел не ловить, тож їх довелося закривати промптом.

Третій висновок — практичний: на демонстраційних даних інструмент за одну команду знайшов те, що в дашборді лабораторної №3 було помітно лише при уважному розгляді окремого графіка. Притік користувачів виріс у чотири рази, і це зростання маскувало зупинку реєстрацій на iOS; декомпозиція внесків показала це одразу — −98,98% у сегменті, який дав −10,1% до загальної зміни, — а локальна модель окремо вказала на це падіння серед того, що перевірити далі. Саме на таких задачах AI-асистент економить аналітику ті 80% рутини, про які йдеться в завданні.